

JSON Output Manager Documentation

1 Introduction

The `json_output_manager.py` module aggregates and writes CTCC results as JSON when `CTCC_OUTPUT_MODE=json`. Each calculation script instantiates `JSONOutputManager` (via `smart_output.CTCC`) calls `add_*` methods to store its results in memory, then calls `save_to_file()` to write an intermediate `json_output_<scenario_id>_<module>.json`. The parent `ctcc.py` then aggregates these files with `load_from_file()`, computes BCR, calls `add_bcr_metrics()` and `calculate_summary()`, and writes the final `ctcc_results_<scenario_id>.json` using `get_json_results()`.

2 Method Overview

2.1 Purpose

1. On init: set `scenario_id` (or auto-generate), load `technical_details` via `load_technical_details()` (uses `json_loaders`), and initialize `costs`, `benefits`, `summary`, `bcr` dicts.
2. Each calculation script calls `add_build_costs()`, `add_row_costs()`, `add_congestion_curtailement()` etc., to store its result dict under the appropriate key.
3. Each script calls `save_to_file(output_dir, module_name)` to write an intermediate JSON file for aggregation.
4. `ctcc.py` calls `aggregate_json_outputs()` which creates a new `JSONOutputManager`, then `load_from_file()` for each intermediate file to merge `costs`, `benefits`, `bcr`; then `add_bcr_metrics()`, `calculate_summary()`, and `get_json_results()` to build the final dict; `ctcc.py` writes `ctcc_results_<id>.json` to disk.

2.2 Calculation Approach

Output only; no cost/benefit formulas. `calculate_summary()` sums stored costs and benefits into `summary`: total capital (build + row + environmental), total operational (insurance + oandm), total energy/emissions (line_loss + emissions + residual exceedance from congestion_curtailement), total risk (wildfire + outage + wildfire_liability), delay costs, and grand total (nominal, AFUDC, PV). Residual exceedance and congestion/curtailement delay costs are taken from `benefits["congestion_curtailement"]` and included in the appropriate totals.

2.3 Inputs and Outputs

Inputs:

- In-memory results from calculation scripts passed to `add_*`() (e.g. `add_build_costs(results)`).
- `scenario_id` (optional at init; auto-generated if not set).
- Technical parameters loaded from `json_loaders` in `load_technical_details()`.
- BCR metrics from `ctcc.py` via `add_bcr_metrics(bcr_results)`.

Outputs:

- Per-script: `save_to_file()` writes `json_output_<scenario_id>_<module_name>.json` to `outputs/` (or given `output_dir`).
- Final: `get_json_results()` returns a dict with `scenario_id`, `timestamp`, `technical_parameters`, `costs`, `benefits`, `summary`, `bcr`; `ctcc.py` writes this to `ctcc_results_<scenario_id>.json`.

2.4 Integration with Other Scripts

- `smart_output.py` — When `CTCC_OUTPUT_MODE=json`, exports `JSONOutputManager` as `CTCCOutputManager`; all calculation scripts use it.
- `json_loaders.py` — `load_project_technical_details()`, `load_physical_details()`, `get_data()` used in `load_technical_details()`.
- `ctcc.py` — `aggregate_json_outputs()` uses `JSONOutputManager` and `load_from_file()`; `write_final_json_output()` uses `add_bcr_metrics()`, `calculate_summary()`, `get_json_results()` then writes the final JSON file.

3 Key Functions

3.1 `JSONOutputManager.__init__(scenario_id=None)`

Sets `scenario_id` (or auto-generated timestamp), loads `technical_details` via `load_technical_details()` and initializes `costs`, `benefits`, `summary`, `bcr` as empty dicts.

3.2 `load_technical_details()`

Loads project and financial data via `json_loaders` and returns a dict of technical parameters (`project_name`, `construction_type`, `ac_dc`, `capacity_mw`, `line_length_miles`, `reconductor-ing`, `old_capacity_mw`, `delay_years`, `project_lifetime_years`, `social_discount_rate`, etc.). On failure returns a minimal dict with an `error` key.

3.3 add_*() methods

Each `add_*()` stores the passed dict under a fixed key: `add_build_costs` → `costs["build"]`, `add_row_costs` → `costs["row"]`, `add_environmental_mitigation` → `costs["environmental"]`, `add_delay_costs` → `costs["delay"]`, `add_insurance_costs` → `costs["insurance"]`, `add_wildfire_liability` → `costs["wildfire_liability"]`, `add_wildfire_costs` → `costs["wildfire"]`, `add_outage_costs` → `costs["outage"]`, `add_oandm_costs` → `costs["oandm"]`, `add_emissions_costs` → `costs["emissions"]`, `add_line_loss_costs` → `costs["line_loss"]`, `add_congestion_curtailement` → `benefits["congestion"]`, `add_revenue` → `benefits["revenue"]`. `add_bcr_metrics(bcr_results)` sets `self.bcr`.

3.4 calculate_summary()

Computes `self.summary` from `self.costs` and `self.benefits`: capital (build + row + environmental), operational (insurance + oandm), energy/emissions (line_loss + emissions + residual exceedance from congestion_curtailement), risk (wildfire + outage + wildfire_liability), delay; then grand total nominal, AFUDC, and PV.

Key logic (excerpt):

```
1 self.summary["total_capital_pv"] = (  
2     build.get("total_pv", 0) + row.get("total_pv", 0) + env.get("  
3         total_pv", 0)  
4 )  
5 # ... operational, risk, energy_emissions (incl. residual from  
6     congestion_curtailement)  
7 self.summary["grand_total_cost_pv"] = (  
8     self.summary["total_capital_pv"] + self.summary["  
9         total_operational_pv"]  
10    + self.summary["total_risk_pv"] + delay.get("total_pv", 0)  
11    + congestion_delay_pv + curtailment_delay_pv  
12    + self.summary["total_energy_emissions_pv"]  
13 )
```

3.5 get_json_results()

Calls `calculate_summary()` if `summary` is empty, then returns `{scenario_id, timestamp, input_mode, technical_parameters, costs, benefits, summary, bcr}`.

3.6 save_to_file(output_dir=None, module_name=None)

Writes current state (`scenario_id`, `timestamp`, `technical_details`, `costs`, `benefits`, `summary`, `bcr`) to `json_output_<scenario_id>_<module_name>.json` (or `json_output_<scenario_id>.json` if `module_name` is omitted). Default `output_dir` is `../outputs` relative to the script directory. Returns the path to the saved file.

3.7 load_from_file(json_file)

Reads the JSON file and merges `state["costs"]`, `state["benefits"]`, `state["bcr"]` into `self.costs`, `self.benefits`, `self.bcr`. Used by `ctcc.aggregate_json_outputs()` to combine per-script outputs.

Key logic:

```
1 with open(json_file, "r") as f:
2     state = json.load(f)
3 if state.get("costs"):
4     for key, value in state["costs"].items():
5         if value is not None:
6             self.costs[key] = value
7 if state.get("benefits"):
8     for key, value in state["benefits"].items():
9         if value:
10            self.benefits[key] = value
11 if state.get("bcr"):
12     self.bcr = state["bcr"]
```

4 Example

Final output structure (from `get_json_results()`):

```
1 {
2     "scenario_id": "20250101_120000",
3     "timestamp": "2025-01-01T12:00:00",
4     "input_mode": "json",
5     "technical_parameters": { "project_name": "...", "capacity_mw":
6         500, ... },
7     "costs": {
8         "build": { "total_nominal": ..., "total_pv": ..., "total_afudc":
9             ... },
10        "row": { ... },
11        "environmental": { ... },
12        "delay": { ... },
13        "insurance": { ... },
14        "wildfire": { ... },
15        "outage": { ... },
16        "wildfire_liability": { ... },
17        "oandm": { ... },
18        "emissions": { ... },
19        "line_loss": { ... }
20    },
21     "benefits": {
22         "congestion_curtailement": { "congestion_benefit_pv": ..., "
23             residual_exceedance_pv": ..., ... },
```

```
21     "revenue": { ... }
22 },
23 "summary": { "total_capital_pv": ..., "grand_total_cost_pv": ...,
24             ... },
25 "bcr": { "bcr_system": ..., "bcr_capital": ..., ... }
```